

Composante Blockchain

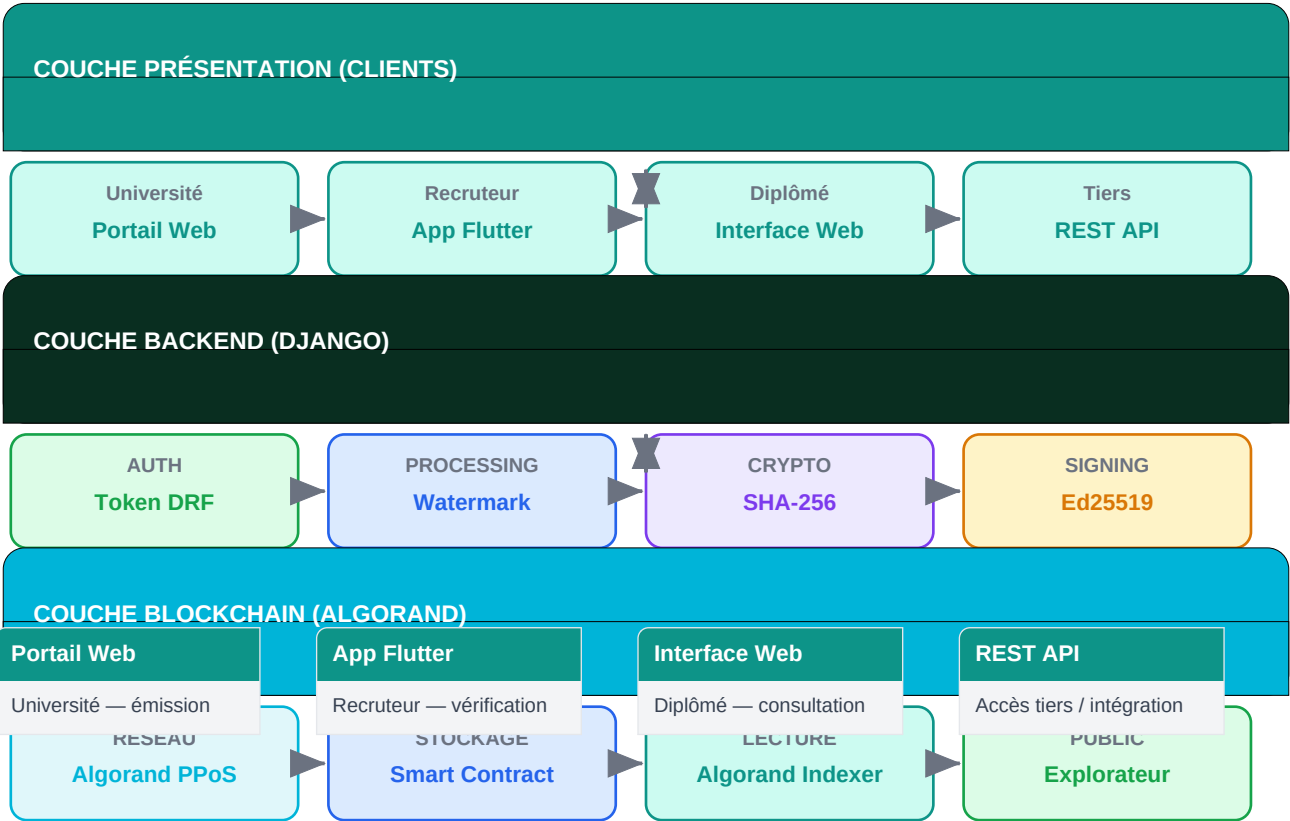
Système de Certification des Diplômes — Burkina Faso

Stack : Algorand PPoS · Django · Ed25519 · SHA-256 · Flutter · Smart Contract

Vue d'ensemble : Ce document décrit la composante blockchain du système de certification numérique de diplômes. L'architecture repose sur trois couches : (1) **Présentation** — portail web université, app Flutter recruteur, interface web diplômé ; (2) **Backend Django** — authentification Token DRF, watermark, hachage SHA-256, signature Ed25519 ; (3) **Blockchain Algorand** — réseau PPoS, stockage via Smart Contract, lecture via Algorand Indexer, explorateur public.

1. Architecture globale — Les 3 couches du système

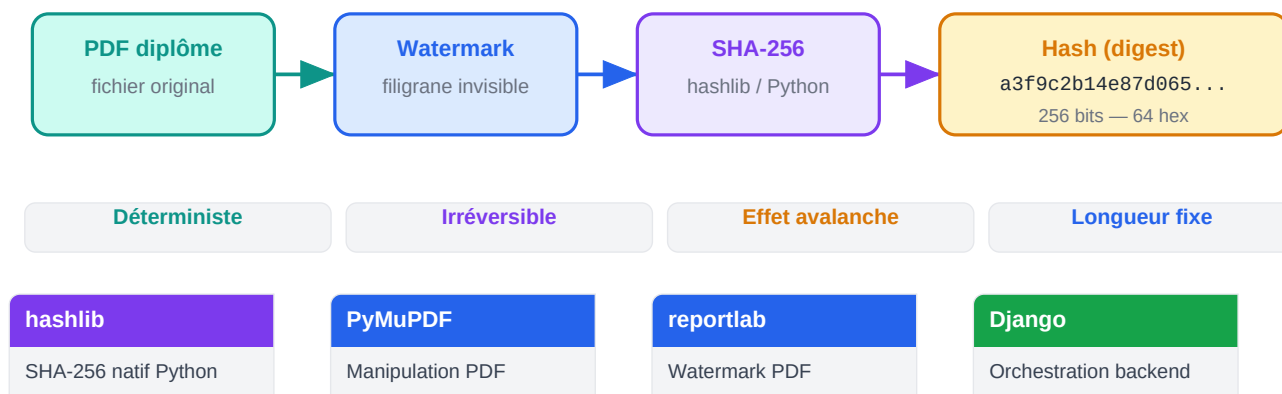
Le système est organisé en trois couches distinctes qui communiquent verticalement. La **couche Présentation** regroupe toutes les interfaces utilisateurs : le portail web de l'université pour émettre les diplômes, l'application **Flutter** du recruteur pour vérifier, et l'interface web du diplômé pour consulter et partager son certificat. La **couche Backend Django** gère l'authentification (**Token DRF**), le traitement du document (**watermark**), le hachage (**SHA-256**) et la signature (**Ed25519**). La **couche Blockchain Algorand** assure le stockage immuable via un Smart Contract, la lecture via l'Algorand Indexer, et la transparence via l'explorateur public.



Django	Token DRF	Algorand PPoS	Smart Contract
Framework backend Python	Auth API sécurisée	Réseau blockchain	Stockage on-chain

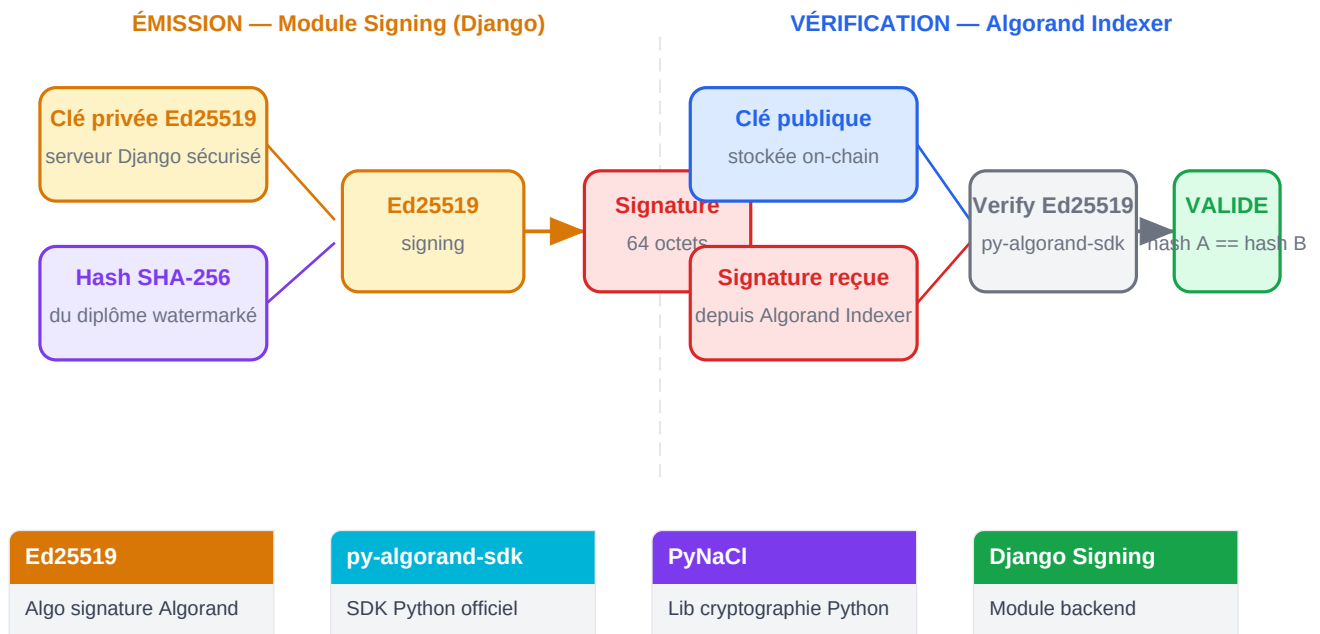
2. Traitement du document — Watermark + SHA-256 (Backend Django)

Lorsque l'université uploadé un PDF, le module **Processing** du backend Django exécute deux opérations avant le hachage. D'abord, un **watermark** (filigrane numérique invisible) est appliqué au document via la bibliothèque Python **reportlab** ou **PyMuPDF**. Ce filigrane encode des métadonnées (identifiant, date, établissement) directement dans le fichier, rendant toute impression non officielle traçable. Ensuite, le module **Crypto** calcule l'empreinte **SHA-256** du document watermarked via **hashlib**. Cette empreinte de 256 bits (64 caractères hexadécimaux) est unique et irréversible — modifier un seul octet du PDF produit un hash totalement différent (effet avalanche).



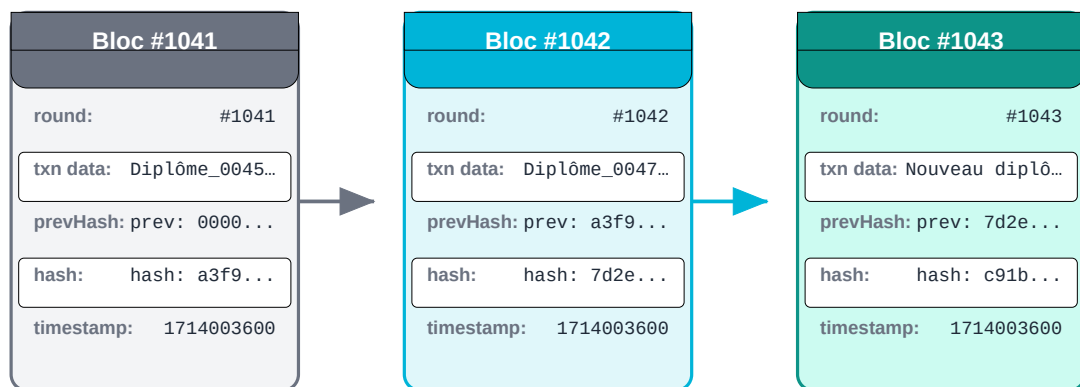
3. La signature numérique — Ed25519 (Module Signing)

Le module **Signing** du backend Django utilise l'algorithme **Ed25519** (Edwards-curve Digital Signature Algorithm, courbe Curve25519). C'est l'algorithme de signature natif d'Algorand — plus rapide et plus compact que ECDSA/secp256k1. Chaque établissement possède une paire de clés : la **clé privée** (conservée secrètement sur le serveur Django) signe le hash SHA-256 du diplôme watermarked, produisant une signature de 64 octets. La **clé publique** est enregistrée on-chain sur Algorand et permet à n'importe qui de vérifier la signature via l'**Algorand Indexer** sans contacter l'établissement. La vérification utilise le SDK officiel **py-algorand-sdk**.

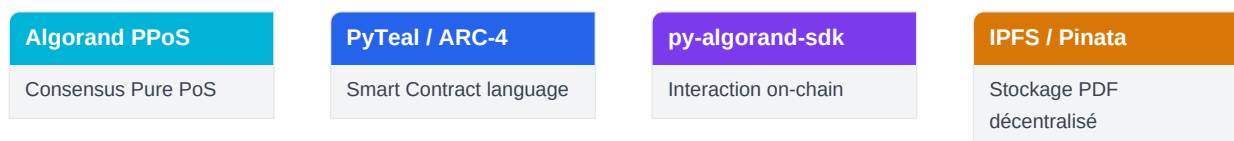


4. La blockchain Algorand — PPoS, Smart Contract, Immuabilité

Algorand est une blockchain publique utilisant le protocole de consensus **Pure Proof of Stake (PPoS)**. Contrairement à Proof of Work (Bitcoin) ou au PoS classique, PPoS sélectionne les validateurs aléatoirement parmi tous les détenteurs d'ALGO de façon cryptographiquement vérifiable, garantissant décentralisation et finalité des transactions en environ **4 secondes**. Le **Smart Contract** (programme stocké on-chain, écrit en **PyTeal** ou **ARC-4 ABI**) reçoit la transaction contenant : le hash SHA-256 du diplôme, la signature Ed25519, la clé publique de l'établissement, l'adresse IPFS (CID) du PDF, et les métadonnées (niveau, domaine, année). Une fois enregistrée, cette transaction est **immuable** : chaque bloc Algorand contient le hash du bloc précédent, rendant toute modification rétroactive détectable et rejetée par le réseau.



Algorand PPoS — Pure Proof of Stake : validateurs sélectionnés aléatoirement, finalité en ~4 secondes



5. Flux d'émission complet — De l'upload au QR Code

Quand l'université uploade un diplôme sur le **Portail Web**, le backend Django déclenche automatiquement une séquence de 5 étapes : (1) réception du PDF et des métadonnées via l'API REST sécurisée par **Token DRF** ; (2) application du **watermark** et calcul du **hash SHA-256** ; (3) **signature Ed25519** du hash avec la clé privée de l'établissement ; (4) envoi d'une transaction au **Smart Contract Algorand** contenant hash + signature + CID IPFS ; (5) génération du **QR Code** contenant le *Transaction ID* (TxID) Algorand, imprimé sur le diplôme et transmis numériquement au diplômé.



Django REST

API d'émission

Token DRF

Authentification école

py-algorand-sdk

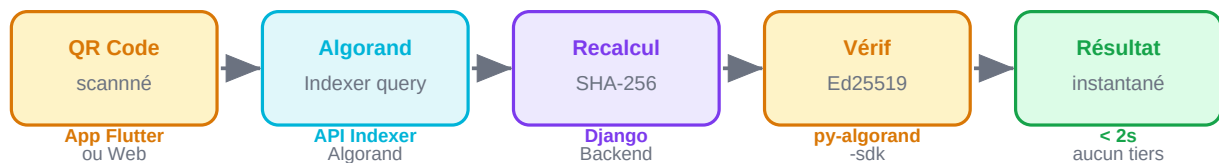
Envoi transaction

qrcode (Python)

Génération QR Code

6. Flux de vérification — De l'App Flutter à la réponse

Le recruteur scanne le QR Code depuis l'**App Flutter**. Le TxID extrait est envoyé à l'**Algorand Indexer** — un noeud de lecture publique qui permet d'interroger la blockchain sans avoir à exécuter un noeud complet. Le backend Django récupère la transaction, recalcule le **hash SHA-256** du PDF présenté et le compare au hash stocké on-chain. Simultanément, il vérifie la **signature Ed25519** avec la clé publique de l'établissement récupérée depuis la blockchain. Si hash et signature sont valides : **diplôme authentique**. Toute l'opération prend moins de 2 secondes, sans aucune interaction avec l'université.



App Flutter

Scanner QR code

Algorand Indexer

Lecture blockchain

py-algorand-sdk

Vérif signature Ed25519

Django API

Orchestration vérif

Pourquoi c'est infalsifiable — Tableau récapitulatif

Tentative de fraude	Mécanisme de défense	Technologie Algorand
Modifier le PDF après certification	Hash SHA-256 recalculé $\neq$ hash on-chain	SHA-256 + Smart Contract
Supprimer le watermark du PDF	Hash change $\rightarrow$ signature invalide immédiatement	SHA-256 + Ed25519
Usurper l'identité d'un établissement	Impossible sans la clé privée Ed25519 de l'école	Ed25519 + py-algorand-sdk
Modifier une transaction on-chain	Hash de bloc change $\rightarrow$ chaîne invalide, rejetée par PPoS	Algorand PPoS
Créer un faux QR Code	TxID inexistant sur Algorand Indexer $\rightarrow$ rejeté	Algorand Indexer

Stack technologique complète

Présentation	Portail Web Université — React.js	App Flutter Recruteur — vérification	Interface Web Diplômé — consultation	REST API Intégration tiers
Backend Django	Django Framework Python	Token DRF Authentification API	PyMuPDF Watermark PDF	hashlib SHA-256 natif
Cryptographie	Ed25519 Signature numérique	py-algorand-sdk SDK Algorand Python	PyNaCl Lib crypto Python	SHA-256 Hachage document
Blockchain	Algorand PPoS Réseau consensus	PyTeal / ARC-4 Smart Contracts	Algorand Indexer Lecture on-chain	IPFS / Pinata Stockage PDF

Note déploiement : Pour la phase de présélection (MVP), la blockchain **Algorand Testnet** sera utilisée — transactions gratuites, environnement identique au Mainnet. Le passage sur **Algorand Mainnet** sera planifié pour la production. Coût d'une transaction Algorand Mainnet : ~ 0.001 ALGO (~ 0.0002 USD).